

Laboratorio 10

1. Plegado de constantes

Se implementa la optimización de plegado de constantes con el propósito de identificar expresiones cuyos operandos corresponden a valores conocidos en tiempo de compilación. Una vez detectadas, dichas expresiones son evaluadas durante la compilación y reemplazadas por su resultado dentro del Árbol de Sintaxis Abstracta (AST). De esta manera, se reduce la cantidad de operaciones que deben ejecutarse en tiempo de ejecución y se simplifica la estructura del árbol. Para almacenar la información asociada a esta optimización, se agregan a la clase base **Exp** los atributos **isConstant** y **constantValue**. El primero indica si una expresión puede ser evaluada durante la compilación, mientras que el segundo almacena el valor calculado.

```
class Exp {  
public:  
    bool isConstant = false;  
    int constantValue = 0;  
};
```

Posteriormente, se crea la clase **ConstantVisitor**, encargada de recorrer el AST y determinar qué expresiones pueden considerarse constantes. Para aplicar la optimización sobre el programa completo, se implementa el método **plegado**, el cual inicia el recorrido del árbol.

```
ConstantVisitor constantVisitor;  
constantVisitor.plegado(program);
```

La clase **ConstantVisitor** redefine los métodos **visit** correspondientes a expresiones, sentencias, declaraciones y funciones del lenguaje. Durante el recorrido, se propaga la información de constantes desde las hojas hacia los nodos superiores, permitiendo identificar expresiones completamente evaluables en tiempo de compilación.

```
class ConstantVisitor : public Visitor {  
public:  
    Program *plegado(Program *program);  
    int visit(BinaryExp *exp) override;  
    int visit(NumberExp *exp) override;  
    int visit(IdExp *exp) override;  
    ...  
};
```

Por ejemplo, la expresión $2 + 3 * 4$ puede evaluarse durante la compilación y reemplazarse por el valor constante 14. Una vez identificadas las expresiones constantes, esta información es utilizada durante la fase de generación de código. En lugar de generar instrucciones para evaluar toda la expresión, se genera una única instrucción que carga directamente el valor calculado en un registro.

```
int GenCodeVisitor::visit(BinaryExp *exp) {  
    if (exp->isConstant) {  
        out << "movq$" << exp->constantValue << ",%rax\n";  
    }  
    return 0;  
}
```

Como resultado, se reduce la cantidad de instrucciones generadas, disminuye el tiempo de ejecución de las expresiones constantes y se produce un código ensamblador más compacto y eficiente.

2. Etiquetado Sethi-Ullman

Se implementa la optimización de etiquetado Sethi-Ullman con el propósito de determinar el número mínimo de registros necesarios para evaluar cada expresión del Árbol de Sintaxis Abstracta (AST). La información obtenida permite seleccionar un orden de evaluación más eficiente durante la generación de código, reduciendo el uso de memoria temporal y la cantidad de operaciones de almacenamiento y recuperación de registros. Para almacenar la etiqueta asociada a cada expresión, se incorpora el atributo `label` en la clase base `Exp`. Dicho atributo contiene el valor calculado por el algoritmo de Sethi-Ullman para cada nodo del árbol.

```
class Exp {  
public:  
    int label = 0;  
};
```

Posteriormente, se crea la clase `SethiUllmanVisitor`, encargada de recorrer el AST y calcular las etiquetas correspondientes para cada expresión. La ejecución de esta optimización se realiza mediante el método `etiquetado`, el cual inicia el recorrido completo del programa.

```
SethiUllmanVisitor sethiUllman;  
sethiUllman.etiquetado(program);
```

La clase `SethiUllmanVisitor` redefine los métodos `visit` necesarios para recorrer las expresiones, sentencias, declaraciones y funciones del lenguaje.

```
class SethiUllmanVisitor : public Visitor {  
public:  
    Program *etiquetado(Program *program);  
    int visit(BinaryExp *exp) override;  
    int visit(NumberExp *exp) override;  
    int visit(IdExp *exp) override;  
    ...  
};
```

Durante el recorrido, se asigna la etiqueta 1 a las expresiones hoja, tales como constantes e identificadores. Para las expresiones binarias, la etiqueta se calcula a partir de las etiquetas de los subárboles izquierdo y derecho. Si ambos subárboles requieren la misma cantidad de registros, la etiqueta del nodo actual se incrementa en una unidad. En caso contrario, se conserva la etiqueta de mayor valor.

```
if (leftLabel == rightLabel)
    exp->label = leftLabel + 1;
else
    exp->label = max(leftLabel, rightLabel);
```

Una vez calculadas las etiquetas, estas se utilizan durante la generación de código para decidir el orden de evaluación de los operandos. Cuando el subárbol izquierdo requiere una cantidad de registros mayor o igual que el derecho, se evalúa primero el izquierdo. En caso contrario, se evalúa primero el derecho. Esta estrategia minimiza el número de accesos a memoria temporal.

```
int GenCodeVisitor::visit(BinaryExp *exp) {
    int l = exp->left ? exp->left->label : 0;
    int r = exp->right ? exp->right->label : 0;
    if (r == 0) {
        exp->left->accept(this);
        out << "movq□%rax,□%rcx\n";
        exp->right->accept(this);
    }
    else if (l >= r) {
        exp->left->accept(this);
        out << "pushq□%rax\n";
        exp->right->accept(this);
        out << "movq□%rax,□%rcx\n";
        out << "popq□%rax\n";
    }
    else {
        exp->right->accept(this);
        out << "pushq□%rax\n";
        exp->left->accept(this);
        out << "movq□%rax,□%rcx\n";
        out << "popq□%rax\n";
        out << "xchgq□%rax,□%rcx\n";
    }
    switch (exp->op) {
        case PLUS_OP:
            out << "addq□%rcx,□%rax\n";
            break;
        case MINUS_OP:
            out << "subq□%rcx,□%rax\n";
            break;
        case MUL_OP:
            out << "imulq□%rcx,□%rax\n";
            break;
    }
    return 0;
}
```

Como resultado, la información proporcionada por las etiquetas Sethi-Ullman permite generar código ensamblador más eficiente, disminuyendo el uso de registros temporales y reduciendo la cantidad de operaciones auxiliares necesarias para evaluar expresiones complejas.